

人生ロングラン・ノベルゲーム

Model 層 実装報告書

担当：志熊 陽斗

2026 年 2 月 15 日

1 はじめに

本レポートでは、Java を用いた「人生ロングラン・ノベルゲーム (Infinite Life Tunnel)」開発において、筆者が担当した **Model 層 (データとロジック)** の実装詳細について解説する。

MVC アーキテクチャにおける Model 層の役割は、View (表示) や Controller (入力制御) から独立し、以下の責務を果たすことである。

- プレイヤーの状態 (年齢、体力、資産など) の管理
- ゲームの進行ルール (イベント分岐、死亡判定) の実行
- データの整合性の保持

本システムでは、複雑なイベント分岐と可変時間システム (選択肢によって経過年数が異なる仕組み) を実現するため、Model 層を複数のクラスに分割して設計した。以下に各クラスの詳細を述べる。

2 Player クラス (状態管理)

Player クラスは、主人公の現在のステータスを保持・管理するクラスである。カプセル化の原則に基づき、フィールドはすべて `private` とし、外部からの不正な書き換えを防ぐ設計としている。

2.1 ソースコードと解説

```
1 class Player {
2     // 定数定義
3     public static final int HUMAN_LIFESPAN = 100;
4
5     // プレイヤーの状態 (フィールド)
```

```

6     private int age;           // 年齢
7     private int health;        // 体力 (0-100)
8     private int stress;        // ストレス (0-100)
9     private long money;        // 所持金 (大きな値を扱うためlong)
10    private boolean isAlive;   // 生存フラグ
11
12    // コンストラクタ
13    public Player() {
14        this.age = 0;           // 0歳からスタート
15        this.health = 50;       // 乳幼児期は体力が低め
16        this.stress = 0;        // ストレスなし
17        this.money = 0;         // 所持金ゼロ
18        this.isAlive = true;    // 生存状態で開始
19    }

```

ソースコード 1 Player.java (定数とフィールド定義)

コード解説 (フィールド定義)

- **2 行目:** HUMAN_LIFESPAN を定数として定義することで、ゲーム終了条件 (100 歳) を変更する際に一箇所を修正するだけで済むようにしている。
- **5-9 行目:** 各ステータス変数を private で宣言している。特に money は将来的に数億円規模になる可能性があるため、int ではなく long 型を採用した。
- **12-18 行目:** コンストラクタにて初期化を行う。本ゲームは「0 歳からの人生」をテーマとしているため、age を 0、health を 50 (成人の半分程度) として、リアリティを持たせている。

```

1     // 年齢を加算するメソッド
2     public void incrementAge(int years) {
3         this.age += years;
4     }
5
6     // 体力を増減させるメソッド
7     public void changeHealth(int amount) {
8         this.health += amount;
9         // 上限キャップ処理
10        if (this.health > 100) this.health = 100;
11    }
12
13    // ストレスを増減させるメソッド
14    public void changeStress(int amount) {
15        this.stress += amount;
16        // 下限キャップ処理
17        if (this.stress < 0) this.stress = 0;

```

```

18     }
19
20     // 所持金を増減させるメソッド
21     public void changeMoney(long amount) {
22         this.money += amount;
23         // 借金（マイナス）も許容するため、下限チェックは行わない
24     }

```

ソースコード 2 Player.java (状態操作メソッド)

コード解説 (状態操作)

- 2-4 行目 (incrementAge) : 本ゲームの特徴である「可変時間システム」に対応するため、固定値 (+1) ではなく、引数 years を受け取って加算するように実装している。
- 7-11 行目 (changeHealth) : 体力が 100 を超えないようにガード処理を入れている。
- 14-18 行目 (changeStress) : ストレスがマイナスにならないように制御している。
- 21-24 行目 (changeMoney) : 住宅ローンなどのイベントを想定し、所持金はマイナス (借金状態) になれる仕様とした。

```

1     // 死亡判定を行うメソッド
2     public void checkVitality() {
3         if (this.health <= 0 || this.stress >= 100) {
4             this.isAlive = false;
5         }
6     }
7
8     // 各フィールドの値を取得するゲッター
9     public int getAge() { return age; }
10    public boolean isAlive() { return isAlive; }
11    public int getHealth() { return health; }
12    public int getStress() { return stress; }
13    public long getMoney() { return money; }
14 }

```

ソースコード 3 Player.java (生死判定とゲッター)

コード解説 (生死判定)

- 2-6 行目 (checkVitality) : ゲームオーバー条件を判定する重要なメソッドである。
 - health <= 0: 体力が尽きた場合 (過労死、病死など)。
 - stress >= 100: ストレスが限界を超えた場合 (精神的崩壊)。
- このメソッドはイベント選択ごとに Controller 経由で呼び出され、生存フラグ isAlive を更新する。

- **9-13 行目:** View 層が画面表示を行うために、現在の値を読み取るだけのメソッド（ゲッター）を提供している。

3 データ構造（Choice クラス・LifeEvent クラス）

本ゲームはノベルゲーム形式であり、イベント（LifeEvent）とそれに紐づく選択肢（Choice）のデータ構造がシステムの根幹を成している。Model 層では、これらのデータをオブジェクト指向の考え方に基づき、独立したクラスとして定義した。

3.1 Choice クラス（選択肢データ）

Choice クラスは、プレイヤーが選択可能な「一つの行動」を表すクラスである。単なるテキストデータだけでなく、その選択を選んだ結果として発生するステータス変動値や、経過時間を保持している。

```
1  class Choice {
2      public String text;          // 選択肢の文言
3      public int healthDelta;     // 体力増減値
4      public int stressDelta;     // ストレス増減値
5      public long moneyDelta;     // 所持金増減値
6      public int ageDelta;        // 経過年数（可変時間システムの核）
7
8      // コンストラクタ
9      public Choice(String text, int h, int s, long m, int ageDelta
10         ) {
11          this.text = text;
12          this.healthDelta = h;
13          this.stressDelta = s;
14          this.moneyDelta = m;
15          this.ageDelta = ageDelta;
16      }
17
18      // UI表示用にテキストを整形するメソッド
19      public String getText() {
20          return text + "␣(+ " + ageDelta + "年)";
21      }
22  }
```

ソースコード 4 Choice.java

コード解説

- **2-6 行目（フィールド定義）**：各フィールドは、その選択肢を選んだ時の「変化量（Delta）」を表す。
 - － `ageDelta`: 本システムの最重要フィールドである。従来のすごろく型ゲームのように「1 ターン = 1 年」と固定するのではなく、選択肢ごとに「大学進学 = 4 年」「就職 = 1 年」「短期バイト = 0 年（数ヶ月）」といった異なる時間を設定することを可能にしている。
- **18-20 行目（`getText` メソッド）**：View 層がボタンを描画する際、プレイヤーに「この選択で何年経過するか」を明示するために、テキストを加工して返すメソッドを用意した。

3.2 LifeEvent クラス（イベントコンテナ）

LifeEvent クラスは、ゲーム内の一つの場面（シーン）を表すクラスである。一つのイベントタイトルに対し、複数の Choice オブジェクトをリスト構造で保持する「1 対多」の関係を持っている。

```
1  import java.util.ArrayList;
2  import java.util.List;
3
4  class LifeEvent {
5      private String title;           // イベントのタイトル・本文
6      private List<Choice> choices;   // 選択肢のリスト
7
8      // コンストラクタ
9      public LifeEvent(String title) {
10         this.title = title;
11         this.choices = new ArrayList<>(); // 空のリストで初期化
12     }
13
14     // 選択肢を追加するメソッド
15     public void addChoice(String text, int h, int s, long m, int
16         ageDelta) {
17         choices.add(new Choice(text, h, s, m, ageDelta));
18     }
19
20     // ゲッターメソッド
21     public String getTitle() { return title; }
22     public String getText() { return title; } // View用
23     public List<Choice> getChoices() { return choices; }
24 }
```

コード解説

- **5-6 行目:** 選択肢を配列 (Choice[]) ではなく List<Choice>で管理している。これにより、イベントによって選択肢が2つだったり4つだったり可変になっても柔軟に対応できる。
- **14-16 行目 (addChoice) :** 外部 (GameLogic) から選択肢を追加するためのメソッド。内部で new Choice(...) を行うことで、Logic 側のコード記述量を減らし、可読性を高めている。
- **Design Pattern:** この構造は「コンポジットパターン」の簡易版とも言え、イベントと選択肢を階層構造で管理することで、将来的に「選択肢の中にさらにイベントが含まれる」といった拡張も容易になっている。

4 ロジック (GameLogic クラス)

GameLogic クラスは、プレイヤーの状態 (年齢) に基づいて、適切なイベントを生成・選択する「ビジネスロジック」を担当するクラスである。条件分岐 (if-else, switch) と乱数 (Math.random) を組み合わせることで、直線的ではない多様な人生シナリオを実現している。

```
1  class GameLogic {
2
3      // メインのイベント振り分けメソッド
4      public LifeEvent getEventForAge(int age) {
5          if (age < 23) {
6              return getChildhoodEvent(age);
7          } else if (age < 65) {
8              return getAdultEvent(age);
9          } else if (age < 100) {
10             return getSeniorEvent(age);
11          } else {
12              return getEndingEvent();
13          }
14      }
```

ソースコード 6 GameLogic.java (イベント振り分け)

コード解説 (イベント振り分け)

- **4-12 行目:** 年齢に応じて呼び出すメソッドを切り替える「ファサード (窓口)」の役割を果たしている。

- 0-22 歳: getChildhoodEvent (子供・学生時代)
- 23-64 歳: getAdultEvent (社会人時代)
- 65-99 歳: getSeniorEvent (老後)
- 100 歳以上: getEndingEvent (エンディング)

このようにメソッドを分割することで、コードの可読性を保ち、将来的なイベント追加（例：青年期の拡張）を容易にしている。

4.1 各年代のイベント生成ロジック

4.1.1 子供・学生時代（特定年齢イベント）

0 歳から 22 歳までは、入学・卒業・成人式など年齢特有のイベントが多いため、switch 文を用いて年齢ごとに固定のイベントを定義した。

```

1      private LifeEvent getChildhoodEvent(int age) {
2          LifeEvent event = new LifeEvent("イベントエラー");
3
4          switch (age) {
5              case 0:
6                  event = new LifeEvent("【0歳】誕生。...");
7                  event.addChoice("大声で泣く", 5, 0, 0, 3); // 3歳へ
8                  event.addChoice("静かにする", 0, 0, 0, 3);
9                  break;
10             case 6: // 小学校入学
11                 event = new LifeEvent("【6歳】小学校入学。...");
12                 event.addChoice("友達を作る", 5, 5, 0, 2); // 8歳へ
13                 // ... 省略 ...
14                 break;
15             case 18: // 進路選択
16                 event = new LifeEvent("【18歳】進路選択。...");
17                 event.addChoice("大学進
18                     学", 0, 10, -5000000, 1); // 19歳へ
19                 event.addChoice("就
20                     職", -5, 30, 2000000, 5); // 23歳へジャンプ
21                 break;
22             // ... 他の年齢 ...
23         }
24         return event;
25     }

```

ソースコード 7 GameLogic.java (子供時代)

コード解説

- 7, 12, 17 行目（経過時間の制御）：選択枝の第 5 引数（ageDelta）により、次に訪れるイベント年齢を制御している。
 - － 0 歳の次は 3 歳（+3 年）、6 歳の次は 8 歳（+2 年）と細かく設定。
 - － 18 歳の選択枝では、「大学進学」なら 1 年後の 19 歳へ進むが、「就職」を選ぶといきなり 5 年後の 23 歳（社会人編）へスキップする。これにより人生の分岐を表現している。

4.1.2 社会人・老後（ランダムイベント）

23 歳以降は長い期間が続くため、年齢による固定イベントではなく、乱数を用いた確率的なイベント発生を実装した。

```
1     private LifeEvent getAdultEvent(int age) {
2         LifeEvent event;
3         double rnd = Math.random(); // 0.0～1.0 の乱数生成
4
5         if (age < 35) { // 若手時代
6             if (rnd < 0.4) {
7                 event = new LifeEvent("【" + age + "歳】仕事のミ
8                     ス...");
9             } else if (rnd < 0.7) {
10                event = new LifeEvent("【" + age + "歳】結婚ラッ
11                    シュ...");
12            } else {
13                event = new LifeEvent("【" + age + "歳】昇進チャン
14                    ス!");
15            }
16        }
17        // ... 中堅・ベテラン時代 ...
18        return event;
19    }
```

ソースコード 8 GameLogic.java（社会人時代）

コード解説

- 3 行目: Math.random() を使用することで、同じ年齢でもプレイするたびに異なるイベントが発生する可能性がある（リプレイ性の確保）。
- 5 行目: 年齢層（若手・中堅・ベテラン）によって発生するイベントプールを切り替えている。これにより「20 代で老後の心配をする」といった不自然な状況を防いでいる。

5 統合とインターフェース (GameModel クラス)

GameModel クラスは、前述した Player (状態) と GameLogic (ロジック) を統合し、Controller 層に対する「単一の窓口 (Facade)」を提供するクラスである。

Controller 層は、このクラスのメソッドのみを呼び出すことでゲームを進行させる。これにより、内部の複雑なデータ構造や計算ロジックが Controller 層に漏れ出すことを防いでいる (疎結合の実現)。

```
1  class GameModel {
2      private final Player player;
3      private final GameLogic logic;
4
5      public GameModel() {
6          this.player = new Player();
7          this.logic = new GameLogic();
8      }
9
10     // 次のイベントを取得する (Logicへ委譲)
11     public LifeEvent nextEvent() {
12         return logic.getEventForAge(player.getAge());
13     }
14
15     // 選択枝の結果を適用する (
16     //     Controllerから呼ばれる主要メソッド)
17     public DeathResult applyChoice(Choice choice) {
18         // 1. ステータス反映
19         player.changeHealth(choice.healthDelta);
20         player.changeStress(choice.stressDelta);
21         player.changeMoney(choice.moneyDelta);
22
23         // 2. 死亡判定
24         player.checkVitality();
25         if (!player.isAlive()) {
26             return new DeathResult(DeathResult.Type.DEAD, "過労
27                 死...", "...", player.getAge());
28         }
29
30         // 3. 年齢加算 (可変時間システム)
31         player.incrementAge(choice.ageDelta);
32
33         // 4. クリア判定 (100歳到達)
34         if (player.getAge() >= Player.HUMAN_LIFESPAN) {
35             return new DeathResult(DeathResult.Type.DEAD, "大往
36                 生", "...", player.getAge());
37         }
38     }
39 }
```

```

34         }
35
36         // 5. 生存継続
37         return new DeathResult(DeathResult.Type.ALIVE, "", "",
                                player.getAge());
38     }
39 }

```

ソースコード 9 GameModel.java (主要メソッド)

コード解説

- **11-13 行目 (nextEvent)** : どのイベントを出すかの判断を GameLogic に完全に委譲 (Delegate) している。Controller は「次のイベントをくれ」と言うだけでよく、年齢による分岐ロジックを知る必要がない。
- **16-37 行目 (applyChoice)** : ゲームの 1 ターンの処理をカプセル化したメソッドである。
 1. **ステータス更新**: 選択枝の効果を Player に反映させる。
 2. **死亡チェック**: 更新後の値で生存確認を行う。
 3. **時間経過**: 生きていれば年齢を進める。
 4. **クリアチェック**: 寿命 (100 歳) に達したかを確認する。

この一連の流れを一つのメソッド内に隠蔽することで、処理順序の間違い (例: 死亡しているのに年齢が増えるなど) を防いでいる。
- **戻り値 (DeathResult)** : 処理結果 (生存か死亡か) を独自のデータクラス DeathResult に詰めて返すことで、Controller 側での分岐処理を簡潔にしている。

6 まとめ

本開発において、Model 層の実装を担当した。特に注力したのは、以下の 2 点である。

1. **可変時間システムの実現**: Choice クラスに ageDelta フィールドを持たせ、選択枝によって「1 年進む」「5 年進む」といった柔軟な時間経過を可能にした。これにより、大学進学や就職といった人生の分岐点をリアルに表現できた。
2. **責務の明確な分離**: 状態管理 (Player)、データ定義 (LifeEvent/Choice)、分岐ロジック (GameLogic)、外部窓口 (GameModel) とクラスを役割ごとに分割した。この設計により、例えば「イベントを増やしたい」場合は GameLogic のみを修正すればよく、他のクラスへの影響を最小限に抑えることができる。

MVC アーキテクチャの Model 層として、堅牢かつ拡張性の高い実装を行えたと考える。